

university of groningen

Performance Engineering: Lecture #08

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-24

Overview



1. Loop Optimizations (2)
2. SIMD and Vector Instructions
3. Auto-Vectorization (1)
4. Final

Content



1. Loop Optimizations (2)

2. SIMD and Vector Instructions

3. Auto-Vectorization (1)

4. Final

Case Study: `mat_accum`



- **Purpose** of this case study: Explore impact of (outer) loop unroll and jam
- **Specification** for `mat_accum`:
  - Implement the following task:

    $$A_{i,j} \leftarrow A_{i,j-1} + b_j$$

    for $i = 0, \ldots, N-1$ and $j = 1, \ldots, M-1$
  - The variables are double-precision numbers

mat_accum: Initial Implementation



```c
#include <stdlib.h>

void kernel(double *restrict a, double *restrict b, int n, int m) {
    for (int i = 0; i < n; i++)
      for (int j = 1; j < m; j++)
          a[i*m + j] = a[i*m + (j-1)] + b[j];
}

int main() {
  const int n = 1024;
  const int m = 2048;
  double *a   = (double *) calloc(n*m, sizeof(double));
  double *b   = (double *) calloc(m, sizeof(double));

  for (int i = 0; i < 1000; i++)
    kernel(a, b, n, m);
}
```

Loop Unroll



- **Loop unroll** = A loop optimization that copies the loop body to reduce the number of iterations
- **Target benefit(s)**
  - Reduce loop control overhead (branching, condition checks)
  - Enable further optimizations within larger basic blocks
- **Possible drawback(s)**
  - Large code size
  - More instruction cache misses

mat_accum: Implementation with Unroll



```c
#include <stdlib.h>

void kernel_unroll( double *restrict a, double *restrict b, int n, int m) {
  for (int i = 0; i < n; i += 2)
    for (int j = 1; j < m; j++)
      a[i*m + j] = a[i*m + (j-1)] + b[j];

  for (int i = 1; i < n; i += 2)
    for (int j = 1; j < m; j++)
      a[i*m + j] = a[i*m + (j-1)] + b[j];
}

int main() {
  const int n = 1024;
  const int m = 2048;
  double *a   = (double *) calloc(n*m, sizeof(double));
  double *b   = (double *) calloc(m, sizeof(double));

  for (int i = 0; i < 1000; i++)
    kernel_unroll(a, b, n, m);
}
```

Question: Why not unroll loop over j?

Loop Carry Dependence



- **Loop carry dependence** = Next loop iteration depends on the previous loop iteration
  - Also known as **loop-carried dependency**
- **Example**:

```
1 double a[N];
2 double s = 0.0;
3 for (int i = 0; i < N; i++) {
4     s = s + a[i];
5 }
```

Loop Unroll and Jam



- **Loop unroll and jam** = An optimization technique where an outer loop is unrolled, then (multiple) inner loops are jammed (fused)

mat_accum: Implementation with Unroll and Jam



```c
#include <stdlib.h>

void kernel_unroll_jam( double *restrict a, double *restrict b, int n, int m) {
  for (int i = 0; i < n; i += 2)
    for (int j = 1; j < m; j++) {
      a[(i+0)*m + j] = a[(i+0)*m + (j-1)] + b[j];
      a[(i+1)*m + j] = a[(i+1)*m + (j-1)] + b[j];
    }
}

int main() {
  const int n = 1024;
  const int m = 2048;
  double *a   = (double *) calloc(n*m, sizeof(double));
  double *b   = (double *) calloc(m, sizeof(double));

  for (int i = 0; i < 1000; i++)
    kernel_unroll_jam(a, b, n, m);
}
```

`mat_accum`: Kunpeng 920 Results

	N_{cycle}	N_{instr}
Original, `-O1`	$2.92 \cdot 10^{10}$	$1.47 \cdot 10^{10}$
Unroll, `-O1`	$2.93 \cdot 10^{10}$	$1.47 \cdot 10^{10}$
Unroll+jam, `-O1`	$1.47 \cdot 10^{10}$	$1.05 \cdot 10^{10}$
Original, `-O3`	$2.53 \cdot 10^{5}$	$1.55 \cdot 10^{5}$
Unroll, `-O3`	$2.21 \cdot 10^{5}$	$1.55 \cdot 10^{5}$
Unroll+jam, `-O3`	$2.12 \cdot 10^{5}$	$1.55 \cdot 10^{5}$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - Using compiler optimization level `-O1`: Out loop unroll does not help, jamming is important
  - Using compiler optimization level `-O3`: Explicit optimization can still be helpful

Case Study: Squared Sums



- **Purpose** of this case study: Explore a strategy to unroll loops with loop-carry-dependences
- **Specification** for `vreduce2`:
  - Implement the following task:

$$s \leftarrow \sum_{i=0}^{N-1} a_i^2$$

  - The variables are double-precision numbers

Squared Sums: Initial Implementation



```c
#include <stdlib.h>

double sqsum(double *restrict a, int n) {
  double s = 0.0;
  for (int i = 0; i < n; i++)
    s += a[i] * a[i];
  return s;
}

double sum;

int main() {
  int n = 8192;
  double *a = (double *)calloc(n, sizeof(double));
  for (int i = 0; i < 10000; i++)
    sum = sqsum(a, n);
}
```

Loop Unroll with Dependency Break



- **Loop unroll with dependency break**: Optimization technique for unrolling loops with loop-carry-dependences
- Option to break dependence: **Partial reductions**

Squared Sums: Unrolled Implementation



```c
#include <stdlib.h>

double sqsum_unrolled(double *restrict a, int n) {
  double s0 = 0.0;
  double s1 = 0.0;
  double s2 = 0.0;
  double s3 = 0.0;
  for (int i = 0; i < n; i += 4) {
    s0 += a[i+0] * a[i+0];
    s1 += a[i+1] * a[i+1];
    s2 += a[i+2] * a[i+2];
    s3 += a[i+3] * a[i+3];
  }
  return (s0 + s1 + s2 + s3);
}
```

The remaining code remains unchanged

Squared Sums: Kunpeng 920 Results



	N_{cycle}	N_{instr}
Original, `-O1`	$3.28 \cdot 10^8$	$4.10 \cdot 10^8$
Unrolled, `-O1`	$1.36 \cdot 10^8$	$2.67 \cdot 10^8$
Original, `-O3`	$4.09 \cdot 10^8$	$3.28 \cdot 10^8$
Unrolled, `-O3`	$1.03 \cdot 10^8$	$1.03 \cdot 10^8$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - Using compiler optimization level `-O1`: Number of cycles and instructions improve by $2.4\times$ and $1.5\times$
  - Using compiler optimization level `-O3`: Compiler is not able to reach similar performance without code modifications

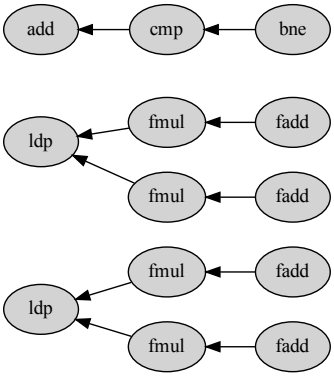
Squared Sums: Unrolled Implementation Assembler



```
8   for (int i = 0; i < n; i += 4) {
9     s0 += a[i+0] * a[i+0];
10    s1 += a[i+1] * a[i+1];
11    s2 += a[i+2] * a[i+2];
12    s3 += a[i+3] * a[i+3];
13  }
```

```
21  .L3:
22          ldp     d30, d31, [x2]
23          fmul    d30, d30, d30
24          fadd    d29, d29, d30
25          fmul    d31, d31, d31
26          fadd    d27, d27, d31
27          ldp     d30, d31, [x2, 16]
28          fmul    d30, d30, d30
29          fadd    d28, d28, d30
30          fmul    d31, d31, d31
31          fadd    d0, d0, d31
32          add     x2, x2, 32
33          cmp     x2, x0
34          bne     .L3
```

Squared Sums: Instruction Dependencies




```
21 .L3:
22         ldp     d30, d31, [x2]
23         fmul    d30, d30, d30
24         fadd    d29, d29, d30
25         fmul    d31, d31, d31
26         fadd    d27, d27, d31
27         ldp     d30, d31, [x2, 16]
28         fmul    d30, d30, d30
29         fadd    d28, d28, d30
30         fmul    d31, d31, d31
31         fadd    d0, d0, d31
32         add     x2, x2, 32
33         cmp     x2, x0
34         bne     .L3
```

Critical Path Analysis



- **Critical path** = Longest sequence of dependent instructions that must be executed sequentially to execute a task
- The critical path determines the **theoretical minimum execution time** for that task
- To compute the critical path the edges between dependent instructions are **weighted by instruction latency**

Case Study: `vadd_condzero`



- **Purpose** of this case study: Explore impact of conditions within loops
- **Specification** for `vadd_condzero`:
  - Implement the following task:

$$
\begin{aligned}
a_i &\leftarrow a_i + b_i \\
b_i &\leftarrow 0 \quad \text{if} \quad c > 0.4
\end{aligned}
$$

  - The variables a_i, b_i, and c are double-precision numbers

vadd_condzero: Initial Implementation



```c
#include <stdlib.h>

void kernel(double *restrict a, double *restrict b, double c, int n) {
  for (int j = 0; j < n; j++) {
    a[j] += b[j];
    if (c > 0.4) b[j] = 0;
  }
}

int main() {
  const int n = 16384;
  double *a   = (double *) calloc(n, sizeof(double));
  double *b   = (double *) calloc(n, sizeof(double));

  for (int i = 0; i < 100000; i++) {
    double c = rand() / (double) RAND_MAX;
    kernel(a, b, c, n);
  }
}
```

Loop Unswitching



- **Loop unswitching** = Optimization technique that moves conditional statements outside of loops when the condition does not change during loop execution
- **Target benefit(s)**
  - Performance benefits (at least for large loops)
  - Enable further optimizations within larger basic blocks
- **Possible drawback(s)**
  - Code duplication
  - Reduced code readability

vadd_condzero: Loop Unswitching



```c
#include <stdlib.h>

void kernel(double *restrict a, double *restrict b, double c, int n) {
  if (c > 0.4)
    for (int j = 0; j < n; j++) {
      a[j] += b[j];
      b[j] = 0;
    }
  else
    for (int j = 0; j < n; j++)
      a[j] += b[j];
}

int main() {
  const int n = 16384;
  double *a   = (double *) calloc(n, sizeof(double));
  double *b   = (double *) calloc(n, sizeof(double));

  for (int i = 0; i < 100000; i++) {
    double c = rand() / (double) RAND_MAX;
    kernel(a, b, c, n);
  }
}
```

`vadd_condzero`: Kunpeng 920 Results



	N_{cycle}	N_{instr}
Original, `-O1`	$5.4 \cdot 10^9$	$17.4 \cdot 10^9$
Unswitched, `-O1`	$3.3 \cdot 10^9$	$6.2 \cdot 10^9$
Original, `-O3`	$4.1 \cdot 10^9$	$12.5 \cdot 10^9$
Unswitched, `-O3`	$2.3 \cdot 10^9$	$6.2 \cdot 10^9$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - Using compiler optimization level `-O1`: Number of cycles and instructions improve by $1.6\times$ and $2.8\times$
  - Using compiler optimization level `-O3`: Compiler is not able to reach similar performance, but explicit optimization still helps ($1.4\times$ faster)

Content



1. Loop Optimizations (2)

2. SIMD and Vector Instructions

3. Auto-Vectorization (1)

4. Final

SIMD and Vector Instructions: Introduction



- **Key idea**: Use a single instructions to perform multiple operations
- **SIMD** = Single Instruction Multiple Data
  - Part of Flynn's taxonomy
- SIMD and vector instructions aim on leveraging data parallelism
  - Early focus area: image processing involving processing of many independent pixels
- Example: vector add with 4 **lanes**

$$\begin{pmatrix} z_0 \\ z_1 \\ z_2 \\ z_3 \end{pmatrix} \leftarrow \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

SIMD versus Vector Instructions



- **SIMD instructions** read input operands and write results in single clock cycle
  - Operand width is limited by hardware constraints
    - Register size
    - Bus update rate
  - Currently, vendors support SIMD instructions with an operand width of **up to** $512\,\text{bit}$
- **Vector instructions** repeat instruction execution multiple times until full vector is processed
  - Vectors can be much larger compared to SIMD operands
  - Vector instructions may process multiple data items per clock cycle

Masking



- **Problem**: What happens if data is not a multiple of the SIMD operand width or vector length?
- **Solution 1**: Scalar **tail loop**
  - Process bulk using SIMD or vector instructions
  - Process remainder using scalar instructions
- **Solution 2**: **Masking**
  - Define a mask where the size is equal to the number of lanes
  - If a lane is masked the result is not written

Gather Load and Scatter Store



- **Gather load** = Load multiple data items from scattered memory locations into a consecutive range of addresses (e.g., a SIMD register)

```
1 for (int i = 0; i < N; i++)
2     x[i] = y[idx[i]];
```

- **Scatter store** = Store multiple data items from a consecutive range of addresses (e.g., a SIMD register) to scattered memory locations

```
1 for (i = 0; i < N; i++)
2     y[idx[i]] = x[i];
```

- Variant: **Structured load** and **structured store** instructions
  - These instructions load or store multiple data items with a fixed stride

SIMD/Vector ISAs



- **x86**
  - Streaming SIMD Extensions (SSE), 128-bit
  - Advanced Vector Extensions (AVX, AVX2), 256-bit
  - AVX-512, 512-bit
- **Arm**
  - NEON, 128-bit
  - Scalable Vector Extension (SVE), 128 to 2048-bit
- **Power**
  - VMX/AltiVec, 128-bit
  - VSX, 128-bit
- **RISC-V**
  - RISC-V (vector ISA)

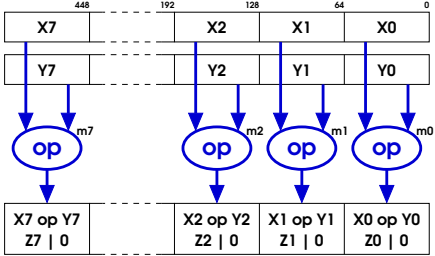
AVX-512 (1/2)

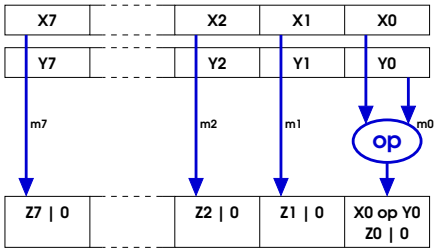


- **AVX-512** refers to a **familiy of instructions**
  - Most recent: Intel Advanced Vector Extensions 10.2 [Intel; 2025]
  - Most widely used encoding: EVEX
  - On Linux see `/proc/cpuinfo` to identify supported ISA
- Number of registers (assuming EVEX):
  - Vector registers: 32 (`ZMM0`, ..., `ZMM31`)
  - Mask registers: 8
- Number of lanes: Depends on data type
  - `double`/64-bit integers: 8
  - `float`/32-bit integers: 16
  - FP16: 32

AVX-512 (1/2)

- AVX-512 supports multi-lane (P=packed) and single-lane (S=scalar) processing





AVX512: Load and Store

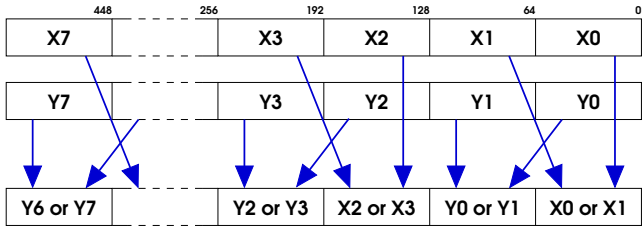


- Relevance of load and store instructions
  - Not all AVX-512 instruction operands can be memory operands
  - Need for gather load and scatter store
- Example instructions:

`VMOVQ xmm1, xmm2/m64`	Move quadword from xmm2/m64 to xmm1
`VMOVQ xmm1/m64, xmm2`	Move quadword from xmm2 to xmm1/m64
`VGATHERQPD zmm1 {k1}, vm64z`	Gather quadword vector
`VSCATTERQPD vm64z {k1}, zmm1`	Scatter quadword vector

AVX512: Shuffle

- `VSHUFPD zmm1{k1}{z}, zmm2, zmm3/m512, imm8`
  Re-order double precision floats depending on immediate value



- Immediate value selects higher (1) or lower (0) part
  - Example: `imm8 = 0x55 = b'01 01 01 01` will select (`Y6,X7,Y4,X5,Y2,X3,Y0,X1`)
- Use case: Swap two elements representing real and imaginary part of double precision complex number

AVX512: Floating-Point Instructions



`VADDPD zmm1 {k1}{z}, zmm2, zmm3/m512/m64bcst{er}`	Add packed double precision floating-point, store result in `zmm1` (writemask `k1`)
`VMULPD zmm1 {k1}{z}, zmm2, zmm3/m512/m64bcst{er}`	Multiply packed double precision floating-point, store result in `zmm1`
`VFMADD132PD zmm1 {k1}{z}, zmm2, zmm3/m512/m64bcst{er}`	Multiply packed double precision floating-point from `zmm1` and `zmm3/m512/m64bscst`, add `zmm2`, store result in `zmm1`

Scalable Vector Extension (SVE/SVE2)



- SVE was introduced as an extension of Arm-v8
- Special feature: **Vector Length Agnostic (VLA)**
  - SIMD operand width can be $128, 256, \ldots, 2048$ bit
  - Compiler does not have to now SIMD operand width at compile time
- Number of registers
  - Vector registers: 32 (`Z0`, ..., `Z31`)
  - Mask/predicate registers: 16 (`P0`, ..., `P15`)

SVE: Load and Store



- Recall: Arm is a load-store ISA
- Example instructions that also support gather/scatter + structured load/store:
  - `LD1D`: Contiguous load doublewords to vector
  - `ST1D`: Contiguous store doublewords from vector
  - `LD2D`: Contiguous load two-doubleword structures to two vectors
  - `ST2D`: Contiguous store two-doubleword structures from two vectors

SVE: Floating-Point Instructions



- `FADD`: Floating-point add vector
- `FADDA`: Horizontal floating-point strictly-ordered reduction (accumulating scalar)
- `FMUL`: Floating-point multiply vectors
- `FMAD`: Floating-point fused multiply-add vectors (predicated), writing multiplicand
- `FCMLA`: Floating-point complex multiply-add with rotate

All instructions support masking/predication

RISC-V “V” Vector Extension (RVV)



- Vector ISA extension with vectors up to $16\,\mathrm{k}$ bit
- RVV defines 32 vector registers of size `VLEN` bits
  - `VLEN` must be a power of 2
- Vectors in RVV are divided in elements
  - The size of elements in bits is at least 8 bits up to `ELEN` bits
- Support of masking
- Today still a few CPUs available supporting RVV
  - Example: SiFive X288 core with vector length up to 4096 bit

Flop per Cycle and Core



- Assuming support of fused multiply-add instructions:

$$B_{\text{fp}} = N_{\text{core}} \cdot N_{\text{SIMD}} \cdot N_{\text{lane}} \cdot 2 \ \frac{\text{Flop}}{\text{cycle}}$$

N_{core}: Number of cores
N_{SIMD}: Number of SIMD pipelines
N_{lane}: Number of SIMD lanes

- Examples for double-precision fused multiply-add instructions:
  - **Intel Golden Cove**: $B_{\text{fp}} = N_{\text{core}} \cdot 2 \cdot 8 \cdot 2 = 32 \cdot N_{\text{core}} \ \frac{\text{Flop}}{\text{cycle}}$
  - **Kunpeng 920**: $B_{\text{fp}} = N_{\text{core}} \cdot 1 \cdot 2 \cdot 2 = 4 \cdot N_{\text{core}} \ \frac{\text{Flop}}{\text{cycle}}$

Content



1. Loop Optimizations (2)

2. SIMD and Vector Instructions

3. **Auto-Vectorization (1)**

4. Final

Compiler Auto-Vectorization



- **Auto-vectorization** = Compiler technique to convert a **scalar implementation** into a **vector implementation**
- Approaches:
  - **Loop-based Vectorization**: Group scalar instructions after loop unrolling
  - **Superword-Level Parallelism (SLP)**: Group similar independent operations together
- Auto-vectorization is performed using intermediate representations
  - GCC: GIMPLE IR

SLP Auto-Vectorization Steps



[Ira Rosen et al.; 2007]

- **Loop unrolling**: Unroll loops
- **Data dependence analysis**: Look for instructions that are not data dependent
- **Clustering**: Identify groups of accesses to adjacent memory addresses to be packed in a vector load
  - Possible use of gather load
  - Note the key importance of data layout
- **Search** for computation chains that can be vectorized
- **Decide** whether to apply vectorization

Auto-Vectorization: Compiler Options



- **GCC**:
  - Enable: `-O3`
  - Reports: `-fopt-info-all` or `-fopt-info-vec-missed`
  - More verbose reports: `-ftree-vectorizer-verbose=3`
- **clang**:
  - Enable: `-O3`
  - Reports:
    - `-Rpass=loop-vectorize`: Identify loops that were successfully vectorized
    - `-Rpass-missed=loop-vectorize`: Identify loops that failed vectorization and indicates if vectorization was specified
    - `-Rpass-analysis=loop-vectorize`: Identify the statements that caused vectorization to fail
  - Tune: `-force-vector-width=<n>`

Memory Alignment (1/2)



- **Memory alignment** = Choice of memory addresses that are multiples of a specific value
  - **Aligned memory address** = memory address that is divisible by a certain value
- Depending on the CPU architecture
  - Alignment is required
  - Special load/store instructions have to be used
  - Performance penalties occur
- Function to test alignment:

```c
#include <stddef.h>
#include <stdint.h>

int is_aligned(const void* ptr, size_t alignment) {
    return ((uintptr_t)ptr % alignment) == 0;
}
```

Memory Alignment (2/2)



- POSIX-compliant aligned memory allocation: `posix_memalign()`

```c
#include <stdlib.h>

double *a;

int main() {
  posix_memalign((void **)&a, 64, 512 * sizeof(double));
  return 0;
}
```

- **Challenge**: How to propagate alignment information?
  - Possible GCC-specific solution: `__builtin_assume_aligned()`

```c
void vmul(double *restrict c, double *restrict a, double *restrict b, int n)
{
  a = __builtin_assume_aligned(a, 64);
  b = __builtin_assume_aligned(b, 64);
  c = __builtin_assume_aligned(c, 64);

  for (int i = 0; i < n; i++) c[i] = a[i] * b[i];
}
```

GCC Pragma `ivdep`



- **Challenge**: The compiler may not be able to determine absence of loop carry dependencies
- **GCC-specific solution**: Use the pragma `ivdep`
  - Programmer asserts that there are no loop carry dependencies
- **Example**:

```
1 void kernel(double *a, double c, int k, int n)
2 {
3 #pragma GCC ivdep
4   for (int i = 0; i < n; i++)
5     a[i] = a[i + k] * c;
6 }
```

  - With this pragma, the programmer asserts that $k \geq 0$

Case Study: Scaling a Vector (`vscale`)



- **Purpose** of this case study: Simple test case for auto-vectorization
- **Specification**: Implement a function that implements

$$\vec{y} \leftarrow \alpha \cdot \vec{x}$$

  using double-precision floating-point numbers

vscale: Scalar Implementation



Benchmark routine:

```
1 void vscale(double *restrict y,
2              double *restrict x,
3              double a, int n) {
4   for (int i = 0; i < n; i++)
5     y[i] = a * x[i];
6 }
```

Benchmark driver:

```
1 #include <stdlib.h>
2
3 void vscale(double *restrict,
4              double *restrict,
5              double, int);
6
7 int main() {
8   int n     = 16384;
9   double *x = (double *) calloc(n, sizeof(double));
10  double *y = (double *) calloc(n, sizeof(double));
11  double a  = 1.0;
12  for (int k = 0; k < 2000000; k++)
13    vscale(y, x, a, n);
14 }
```

vscale: Auto-Vectorized



Compilation using `gcc -O3 -march=native`

Benchmark routine:

```
1 void vscale(double *restrict y,
2              double *restrict x,
3              double a, int n) {
4   for (int i = 0; i < n; i++)
5     y[i] = a * x[i];
6 }
```

Assembler:

```
6  vscale:
7  .LFB0:
8          .cfi_startproc
9          endbr64
10         movq    %rdi, %rcx
11         testl   %edx, %edx
12         jle     .L17
13         leal    -1(%rdx), %eax
14         cmpl    $2, %eax
15         jbe     .L8
16         movl    %edx, %edi
17         vbroadcastsd    %xmm0, %ymm2
18         xorl    %eax, %eax
19         shrl    $2, %edi
20         salq    $5, %rdi
21         .p2align 5
22         .p2align 4,,10
23         .p2align 3
24 .L4:
25         vmulpd  (%rsi,%rax), %ymm2, %ymm1
26         vmovupd %ymm1, (%rcx,%rax)
27         addq    $32, %rax
28         cmpq    %rdi, %rax
29         jne     .L4
```

`vscale`: Results on Intel i7-1165G7



	N_{cycle}	N_{instr}
`-O1`	$4.9 \cdot 10^{10}$	$19.7 \cdot 10^{10}$
`-O3`	$2.8 \cdot 10^{10}$	$9.8 \cdot 10^{10}$
`-O3 -march=native`	$2.2 \cdot 10^{10}$	$4.1 \cdot 10^{10}$

- **Compiler**: GCC 14.2.0
- **Observations**:
  - Latency and number of instructions reduces by about $2.2\times$ and 4.8, respectively, comparing `-O1` and `-O3 -march=native`
- **Crosschecks**:
  - Benchmark is repeated $2 \cdot 10^6\times$ using $N = 16384$ ☞ 0.7 cycles per array element ✔

Content



1. Loop Optimizations (2)

2. SIMD and Vector Instructions

3. Auto-Vectorization (1)

4. Final

Summary Lecture #08



- **Loop optimizations**:
  - Loop unroll, loop unroll and jam, loop unswitching
  - Loop-carry-dependencies and a strategy to break such dependencies
  - Instruction dependency graphs and critical path analysis
- **SIMD and vector instructions**: AVX-512
- Extended overview of **SIMD and vector instructions**: SVE/SVE2, RVV
- **Auto-vectorization**:
  - Compiler auto-vectorization algorithms, options, and reports
  - Possible issues: memory alignment, loop carry dependence
  - Case study: vector scale

Questions?



[Wiki Media]